Acceptance Dynamics Across Cognitive Domains in Speculative Decoding

Saif Mahmoud
College of Engineering, Al Ain University
Abu Dhabi, United Arab Emirates
contact@saifmb.com

***Abstract*—Speculative decoding accelerates large language model (LLM) inference. It uses a small draft model to propose a tree of future tokens. A larger target model then verifies these tokens in a single batched forward pass. Despite the growing body of work on speculative methods, the *degree to which the cognitive characteristics of a task affect acceptance probability* remains largely unexplored. We present an empirical study of tree-based speculative decoding acceptance dynamics. Our study spans four well-established NLP benchmark domains: code generation, mathematical reasoning, logical reasoning, and open-ended chat. For this, we use `TinyLlama-1.1B` as the draft model against `Llama-2-7B-Chat-GPTQ` as the target. Over 99,768 speculative nodes collected from 200 prompts, we derive per-domain acceptance rates, expected accepted lengths, depth-acceptance profiles, and entropy–acceptance correlations. We find that task type is a stronger predictor of acceptance than tree depth. Furthermore, only the chat domain consistently yields an expected accepted length exceeding 1.0 token per step. We also show that the entropy–acceptance correlation is consistently negative but weak across all domains ($\rho \in [-0.20, -0.15]$). Counterintuitively, chat produces the highest entropy yet the highest acceptance rate. We attribute this divergence to the lexical predictability of RLHF-aligned register. These findings have direct implications for domain-aware speculation budgets and draft-model selection strategies.**

***Index Terms*—speculative decoding, large language model inference, tree attention, draft model, acceptance probability, LLM efficiency**

I. Introduction

Autoregressive generation in transformer-based LLMs proceeds one token at a time. Each forward pass through a large model is memory-bandwidth bound rather than compute bound. Therefore, the latency of producing N tokens scales linearly with N and with model size. This makes the interactive deployment of 7B-parameter and larger models challenging on commodity hardware.

Speculative decoding [1] addresses this bottleneck. It observes that a cheap draft model can propose candidate token sequences. A larger, more accurate target model can then verify these sequences in parallel. The expected number of tokens accepted per target forward pass—the *expected accepted length*, $E[L]$—directly governs the wall-clock speedup: if $E[L] > 1$, the target model effectively generates more than one token per invocation, reducing total latency.

The seminal proposal evaluated acceptance under relatively homogeneous conditions. Subsequent work has extended speculative methods to tree-shaped drafts [2, 3]. In these approaches, the draft model proposes a branching set of future hypotheses rather than a single sequence. This further increases the theoretical ceiling on $E[L]$. In a tree draft of depth d and maximum branching factor b, up to b^d candidates are verified in a single target pass.

A critical practical question that has received less attention is: *to what extent does task type determine acceptance probability?* If the gap between domains is substantial, practitioners should tailor speculation budgets on a per-task basis. This includes adjusting draft depth, branching factor, and even draft model selection, rather than applying a universal configuration. Answering this question empirically motivates the present work.

We make the following contributions:

- A reproducible, end-to-end benchmark of tree-based speculative decoding acceptance across four cognitive domains, totalling 99,768 node-level observations.
- Quantification of per-domain expected accepted length, showing only chat exceeds $E[L] = 1.0$ under the present configuration ($E[L]_{\text{chat}} = 1.065$).
- Empirical characterisation of the depth–acceptance relationship, revealing a small but consistent *positive* trend with depth that contradicts the naive expectation of decay.
- Analysis of the entropy–acceptance correlation and the *chat paradox*: the highest-entropy domain simultaneously exhibits the highest acceptance rate, suggesting that token-level lexical predictability and semantic-distributional entropy are orthogonal.

II. Related Work

A. Speculative Decoding

Leviathan et al. [1] and Chen et al. [4] independently proposed the core speculative decoding framework. A draft model generates K tokens autoregressively; the target model scores all $K + 1$ positions in a single forward pass; tokens are accepted token-by-token via a rejection sampling scheme that preserves the target distribution exactly.

B. Tree-Based Extensions

Miao et al. [2] and Cai et al. [3] extend the linear draft to a tree structure, enabling the target model to evaluate exponentially many candidate continuations per forward pass. Tree attention [2] modifies the self-attention mask. This ensures that each leaf's context is isolated from sibling branches. The expected accepted length for a tree draft satisfies

$E[L] = \sum_{d=1}^{D} \prod_{k=1}^{d} \alpha_k$ where α_k is the acceptance probability at depth k.

C. Acceptance Probability Analyses

Prior analytical work [1, 5] establishes that acceptance rate depends on the KL-divergence between draft and target distributions. Zhou et al. [6] show that distilling the target into the draft increases acceptance. To our knowledge, no prior work provides systematic cross-domain acceptance measurements using an off-the-shelf draft model at tree depth > 1.

D. Efficiency Benchmarking

Benchmarks such as MTBench [7] and HumanEval [8] evaluate generation *quality*. However, they do not evaluate the inference-efficiency characteristics of speculative methods across task types. Our work is orthogonal to these. We focus on acceptance physics rather than output correctness.

To our knowledge, no prior work systematically characterizes acceptance dynamics across cognitively distinct task domains. Domain-adaptive speculative decoding has been studied in the context of technical specialization. Examples include function calling, biology, and domain-specific language models. In these cases, the draft model is fine-tuned to match a specialized target [9]. These works focus on closing the distribution gap through training. They do not characterize how cognitive task type affects acceptance under a fixed, general-purpose draft model. Our work is orthogonal. We hold the draft-target pair constant and vary the task domain. This isolates task type as an independent variable in acceptance dynamics.

III. Methodology

A. Models

We select a draft–target pair representing a practical deployment scenario. **Target:** `TheBloke/Llama-2-7B-Chat-GPTQ` [10], a 4-bit GPTQ-quantized variant of Llama-2-7B-Chat, fitting on a 16 GB GPU. **Draft:** `TinyLlama/TinyLlama-1.1B-Chat-v1.0` [11], a 1.1B-parameter model on the Llama-2 architecture. Its shared vocabulary makes it a natural draft candidate.

B. Tree Construction

At each generation step we build a draft tree as follows:

1) The draft model performs a full forward pass over the current context with top-k sampling at depth 1, producing root candidates.
2) For each node at depth $d < D_{\max}$, the draft model scores the path from the root to that node concatenated to the current context. It then expands the top-$b_{\max}$ children.
3) This yields at most $N_{\max}$ nodes per tree.

All forward passes use `use_cache=False`. This ensures the tree construction is independent of any KV-cache state, since tree-branching invalidates a linear cache. The entire frontier per depth level is evaluated as a single batched tensor call. This avoids per-node sequential overhead.

C. Acceptance Scoring

After tree construction, the target model scores all tree leaves in a single batched forward pass. This pass also includes the empty-prefix row. The greedy next token is simultaneously extracted from this row, eliminating one otherwise redundant target call per generation step.

For each node n with token t_n, draft probability $p^{\text{draft}}(t_n)$, and target probability $p^{\text{target}}(t_n)$, the acceptance probability is:

$$\alpha_n = \min\left(1,\ \frac{p^{\text{target}}(t_n)}{p^{\text{draft}}(t_n)}\right) \tag{1}$$

This is the standard speculative rejection-sampling rule [1].

Target entropy at node n is computed from the full target distribution:

$$H_n = -\sum_{v} p^{\text{target}}(v) \log p^{\text{target}}(v) \tag{2}$$

D. Benchmark Datasets

Code — `openai/humaneval` [8]: the canonical Python programming task benchmark. Prompts are function signatures with docstrings; the model is expected to complete the body.

Math — `EleutherAI/hendrycks_math` (algebra subset) [12]: competition-level algebra problems requiring multi-step symbolic reasoning.

Reasoning — `openai/gsm8k` [13]: grade-school arithmetic word problems requiring chain-of-thought numeric reasoning.

Chat — `HuggingFaceH4/ultrachat_200k` [14]: multi-turn instruction-following dialogue drawn from a large-scale RLHF dataset, representing open-ended conversational generation.

Fifty prompts per domain are sampled; generation is capped at 64 tokens. All prompts are truncated to 512 tokens to bound the $O(N^2)$ attention complexity during tree evaluation.

E. Tree Hyperparameters

Hyperparameter	Value
Max tree depth $D_{\max}$	3
Max branches per node $b_{\max}$	2
Top-k (root)	3
Max nodes per tree $N_{\max}$	8
Temperature	0 (greedy)
Max generated tokens	64

IV. Experimental Setup

All experiments are executed on a Kaggle Notebook instance with two NVIDIA Tesla T4 GPUs (15.6 GB VRAM each). The target model is pinned to `cuda:0` and the draft model to `cuda:1`, using `gptqmodel` 6.0.3 and `transformers` 5.5.3 under PyTorch 2.11.0+cu130. The GPTQ kernel selected at runtime is `TritonV2QuantLinear`. All results are reproducible with a fixed seed of 42.

At load time, the target model occupies 4.85 GB of VRAM on `cuda:0`, and the draft model occupies 2.20 GB on `cuda:1`, leaving substantial headroom for batched activation buffers

during tree scoring. Pinning the two models to separate physical GPUs eliminates cross-device attention tensor transfers during the tree-construction inner loop, which would otherwise serialize under slow NVLink-absent PCI-e bus bandwidth. The total experiment runtime across 200 prompts at the reported hyperparameters was approximately 3.5 hours.

V. Results

A. Summary Statistics

Table I reports per-domain acceptance rates and target entropy. A total of 99,768 speculative node observations are collected across 200 prompts.

TABLE I
PER-DOMAIN SPECULATIVE NODE STATISTICS. α: TOKEN-LEVEL ACCEPTANCE PROBABILITY (EQ. 1). H: MEAN TARGET ENTROPY (NATS).

Domain	Nodes	$\bar{\alpha}$	σ_α	$\bar{H}$
Chat	24,592	0.5650	0.4415	1.2517
Code	25,600	0.5382	0.4536	0.8858
Math	25,392	0.5181	0.4501	1.1513
Reasoning	24,184	0.5321	0.4517	0.8943
All	**99,768**	**0.5384**	**0.4500**	**1.0458**

All domains exhibit acceptance rates near 0.5. This reflects the fundamental difficulty for a 1B model to track the token-level distribution of a 7B model. However, there is a 10% relative gap between chat (0.565) and math (0.518). This is practically significant given its cubic amplification inside the expected accepted length formula.

B. Expected Accepted Length

The expected accepted length $E[L]$ is the key operational metric for assessing inference speedup. As defined by the cumulative chain probability [2]:

$$E[L] = \sum_{d=1}^{D} \prod_{k=1}^{d} \bar{\alpha}_k \tag{3}$$

TABLE II
EXPECTED ACCEPTED LENGTH PER DOMAIN. VALUES > 1.0 INDICATE THE SPECULATION YIELDS ON AVERAGE MORE THAN ONE ACCEPTED TOKEN PER TARGET CALL.

Domain	$E[L]$	Speedup Regime
Chat	1.065	**Positive**
Code	0.975	Marginal
Reasoning	0.956	Marginal
Math	0.914	Negative

Table II reveals a clear stratification. Only chat surpasses the $E[L] = 1.0$ threshold. This means speculative decoding is expected to reduce total target forward passes for conversational generation. Math sits below 0.92. This indicates that the overhead of running the draft model and the tree construction erodes any per-pass gain under the current configuration.

The cumulative chain probability decays geometrically with depth for all domains (Table III). At depth 3, chat retains 18.4% probability of a fully accepted chain, compared to only 13.9% for math.

TABLE III
PROBABILITY OF A FULLY ACCEPTED CHAIN OF LENGTH d (I.E., ALL d NODES ON THE ROOT-TO-LEAF PATH ARE ACCEPTED).

Domain	$d = 1$	$d = 2$	$d = 3$
Chat	0.567	0.313	0.184
Code	0.533	0.287	0.156
Reasoning	0.526	0.280	0.151
Math	0.510	0.265	0.139

C. Depth–Acceptance Profile

A natural hypothesis is that acceptance probability decays with tree depth. This is because nodes deeper in the tree condition on increasingly uncertain draft-generated prefixes. Our data refutes this.

TABLE IV
MEAN ACCEPTANCE RATE $\bar{\alpha}$ AT EACH TREE DEPTH, PER DOMAIN.

	Depth			
Domain	1	2	3	$\Delta_{1\to3}$
Chat	0.567	0.553	0.588	$+0.021$
Code	0.533	0.538	0.544	$+0.011$
Math	0.510	0.519	0.525	$+0.015$
Reasoning	0.526	0.532	0.538	$+0.012$

Across all four domains, $\bar{\alpha}$ *increases* slightly from depth 1 to depth 3 (Table IV). This is small in absolute magnitude (+0.011 to +0.021). However, it is consistent across domains and supported by approximately 6,000–13,000 observations per cell.

We interpret this as a *context-commitment effect*: at depth 1, the draft must predict the next content word from an open distribution—the hardest prediction task. Once a syntactic and semantic direction is committed at depth 1, the subsequent tokens in the same chain are more constrained (e.g., the completion of a phrase, the end of a numeric expression). Both the draft and target model agree on that completion more readily. The depth-1 node represents the *semantic pivot*, while depth-2 and depth-3 nodes are increasingly syntactic complements.

D. Position-Within-Generation Effects

Beyond tree depth and task type, we examine whether the position of a generation step within the output sequence affects acceptance. We partition token positions into two equal bins. These are early positions (bin 0, tokens 1–32) and late positions (bin 1, tokens 33–64). We report $\bar{\alpha}$ per (depth, bin) cell in Table V.

At depths 1 and 3, acceptance is consistently higher for tokens generated later in the sequence. This effect is most pronounced at depth 1 (+0.028), where the output has already established a topic, style, and syntactic register. Once the model has committed to a generation direction at earlier positions,

TABLE V
MEAN ACCEPTANCE RATE BY TREE DEPTH AND POSITION BIN (0 = EARLY TOKENS, 1 = LATE TOKENS), AGGREGATED ACROSS ALL DOMAINS.

Depth	Early (bin 0)	Late (bin 1)	Δ
1	0.520	0.548	$+0.028$
2	0.537	0.534	-0.003
3	0.541	0.556	$+0.015$

the marginal next-token prediction task at depth 1 becomes easier for both models. This increases the probability that the draft's choice coincides with the target's preference.

The depth-2 row is a near-zero exception (-0.003). This is consistent with the interpretation that depth-2 nodes are mid-chain syntactic complements. Their predictability is largely determined by their parent node rather than by macro sequence position. The position effect is therefore primarily a property of free-choice token slots (depth 1) rather than constrained completion slots (depth 2).

E. Entropy–Acceptance Correlation

To test whether target model uncertainty predicts draft rejection, we compute the Spearman correlation coefficient ρ between H_n and α_n for each domain.

TABLE VI
SPEARMAN ρ BETWEEN TARGET ENTROPY H_n AND ACCEPTANCE PROBABILITY α_n AT THE NODE LEVEL.

Domain	$\rho(H, \alpha)$
Chat	-0.202
Code	-0.194
Math	-0.186
Reasoning	-0.149

All correlations are negative (higher entropy $\rightarrow$ lower acceptance). They are also statistically significant at $p < 0.001$ given the node-level sample sizes. However, the magnitude is uniformly weak ($|\rho| < 0.21$). This indicates that entropy is a *necessary but not sufficient* predictor of acceptance. The dominant predictor in this case is the hard probability ratio $p^{\text{target}}/p^{\text{draft}}$ under the specific token proposed. It is not the marginal uncertainty of the target distribution.

F. The Chat Paradox

The most striking finding in Table I is that chat exhibits *simultaneously* the highest target entropy (1.252 nats) and the highest acceptance rate (0.565). Code and reasoning have lower entropy (≈ 0.89 nats) yet lower acceptance. Math occupies an intermediate entropy position (1.151 nats) but achieves the lowest acceptance.

These observations jointly refute a simple entropy-determines-acceptance narrative. We propose the following explanation. Target entropy measures the *diversity* of the target model's next-token distribution, not the degree of agreement between draft and target. In chat, the target model spreads probability mass over many semantically valid continuations. This produces high entropy. However, the particular tokens it favours are stylistically common conversational tokens. The draft model has also learned to preferentially predict these tokens (e.g., filler tokens, discourse markers, formulaic response openers). The result is that p^{draft} and p^{target} peak at the same tokens even though both distributions are broad.

In contrast, mathematical text has moderate entropy but imposes strong constraints on *which specific* tokens are numerically correct. The target model places high probability on a small set of numeric and symbolic tokens. The draft model lacks the arithmetic precision of a 7B model. Thus, it selects these tokens less reliably. Low token overlap between the two models' distributions drives low acceptance despite moderate entropy.

VI. DISCUSSION

Practical implications. Our results suggest that the quality of speculation is fundamentally coarse-grained by task type. Practitioners operating speculative decoding in mixed-use inference systems should not apply a uniform draft budget. An example is an API serving both chat and mathematical queries. For chat workloads, deeper trees (larger $D_{\max}$) and relaxed acceptance thresholds are well-justified. In contrast, for mathematical workloads, a shallower tree or a domain-specifically fine-tuned draft model is preferable. This reduces wasted draft computation.

Draft model selection. Given the moderate but consistent acceptance gap, replacing TinyLlama with a maths-specialised 1B draft is a direct and actionable recommendation. An example is a model fine-tuned on MATH or DeepMind's MathMix data. The shared-vocabulary constraint between draft and target remains important. A model fine-tuned on math with the same Llama-2 tokenizer would likely close the $E[L]$ gap substantially.

Request routing. Our results suggest a complementary strategy that requires no re-training. It involves classifying incoming requests by task type before deciding whether to activate speculative decoding at all. A lightweight prompt classifier, even a rule-based heuristic, can segregate chat and conversational requests. For these, $E[L] > 1$ is confirmed. It can separate them from mathematical or code requests, where the current draft provides no net speedup. Disabling speculation for the low-$E[L]$ tail provides a benefit. It eliminates the cost of draft tree construction on requests where it actively hurts latency, all without modifying the models themselves. This approach is trivially composable with existing inference-serving infrastructure. It represents an immediately deployable recommendation from our findings.

VII. LIMITATIONS.

The experiments are conducted with greedy sampling (temperature 0). This represents a maximum acceptance condition for the rejection-sampling criterion. Stochastic sampling would uniformly reduce $E[L]$. The tree hyperparameters, depth 3 and branching factor 2, are mod modest. Deeper trees may surface stronger depth-acceptance trends. All measurements

are at the token level. Output quality, such as pass@k for code or accuracy for math, is outside the scope of this study. Finally, a single draft–target pair is evaluated. Generalisation across other model families requires further study.

VIII. Conclusion

We have presented a systematic empirical analysis of tree-based speculative decoding acceptance dynamics across four NLP benchmark domains. Our principal findings are as follows. First, task type is a stronger determinant of acceptance rate than tree depth. Second, chat is the only domain where the expected accepted length exceeds 1.0 under the evaluated configuration. This represents a genuine latency reduction. Third, acceptance probability exhibits a small positive trend with depth. This is consistent with a context-commitment effect that reduces prediction difficulty at deeper nodes. Finally, entropy–acceptance correlations are negative and statistically significant but weak. Additionally, the high-entropy chat paradox demonstrates that lexical predictability and semantic entropy are orthogonal axes.

These findings motivate domain-adaptive speculation strategies. They also motivate domain-specific draft model selection as first-class design decisions for speculative inference pipelines.

References

[1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 19274–19286. PMLR, 2023.

[2] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, volume 3, pages 932–949. ACM, 2024.

[3] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. MEDUSA: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[4] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[5] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

[6] Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. DistillSpec: Improving speculative decoding via knowledge distillation. *arXiv preprint arXiv:2310.08461*, 2024.

[7] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[9] Jian Hu, Seungyeon Kim, Dheevatsa Mudigere, Maxim Naumov, Jongsoo Park, and Mikhail Smelyanskiy. Training domain draft models for speculative decoding: Best practices and insights. *arXiv preprint arXiv:2503.07807*, 2025. ICLR 2025 Workshop on Sparsity in Computational Optimization (SCOPE).

[10] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. 2023.

[11] Peiyuan Zhang, Guangtao Zeng, Tianhao Wang, and Wei Lu. TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.

[12] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[13] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[14] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. *arXiv preprint arXiv:2305.14233*, 2023.